

MOODLY

Relatório técnico final

Deuwer Rabelais (20241521)

Isimar Quixito (20242038)

Miguel Almeida (20240206)

Vivandro Kambanza (20241805)

Índice

1.Introdução	4
2.O que é o Moodly	5
2.1. Funcionalidades	5
3.Arquitetura do Sistema	5
3.1. Tecnologias	6
3.2. Estrutura de Ficheiros e Routing	6
3.3. Base de dados	6
4.Algoritmos e Estruturas de Dados	8
4.1. Algoritmo de Matching por Interesses.....	8
4.2. Dashboard Estatístico	9
4.3. Visualização de Informação Georreferenciada (SIG).....	10
4.5. Algoritmos por Perfil de Utilizador	10
5. Funcionalidades Implementadas.....	11
5.1. Autenticação e Perfil.....	11
5.2. Descoberta e Conexões.....	11
5.3. Mensagens Diretas.....	11
5.4. Eventos	11
5.5. Administração e Backoffice.....	11
6. Evolução entre Entregas	12
Fase 1 para Fase 2: Principais Alterações.....	12
Fase 2 para Fase 3: Principais Alterações.....	13
7. UI/UX e Design System	14
8. Testes de Usabilidade	14
9. Análise Estatística.....	15
10. Reflexão Crítica	16
11. Divisão de Tarefas	17
12. Conclusão	18
13. Anexos	19
[Anexo 1].....	19
[Anexo 2].....	20
.....	20
[Anexo 3].....	21

[Anexo 4].....	22
----------------	----

1.Introdução

O presente documento constitui o Relatório Técnico Final do projeto Moodly-web, desenvolvido no âmbito da Unidade Curricular de Projeto de Desenvolvimento Web, integrada no 4.º semestre da Licenciatura em Engenharia Informática do IADE.

O projeto seguiu a metodologia das disciplinas do quarto semestre, articulando as contribuições das UCs de Programação Web, Interfaces e Usabilidades, Estatística, Sistemas de Informação Geográficos e Algoritmos e Estruturas de Dados. O Moodly é uma rede social web orientada a estudantes universitários, cujo propósito central é ligar pessoas com interesses culturais comuns como música, videojogos e cinema/séries.

A plataforma permite descobrir utilizadores compatíveis através de um algoritmo de scoring por afinidade, estabelecer ligações, trocar mensagens diretas, organizar eventos geolocalizados e consultar um painel de estatísticas descritivas reservado ao perfil de administrador. Este relatório documenta a totalidade do trabalho realizado ao longo das três fases do projeto, descrevendo a arquitetura técnica, os algoritmos implementados, as decisões de design e as principais diferenças entre as entregas intermédias e a versão final.

2.O que é o Moodly

O Moodly foi concebido para responder a uma necessidade real de socialização entre estudantes que partilham gostos culturais específicos, mas que, fora dos contextos académicos formais, não dispõem de mecanismos para se encontrar. A aplicação distingue-se de redes sociais genéricas por focar a descoberta exclusivamente em sub-interesses culturais granulares como bandas, videojogos ou séries concretas.

2.1. Funcionalidades

Descoberta por afinidade: Algoritmo de scoring que ordena utilizadores por sub-interesses partilhados e reciprocidade

Sistema de conexões: Pedidos de ligação com estados pendente / aceite / recusado

Mensagens diretas: Chat 1-para-1 entre utilizadores conectados, com polling assíncrono

Chat de grupo de evento: Canal de grupo associado a cada evento, partilhado pelos participantes confirmados

Eventos geolocalizados: Criação de eventos com localização selecionada em mapa Leaflet; visualização global em

Backoffice de administração: Gestão de utilizadores, resolução de reports, promoção de admins, dashboard estatístico

Dashboard estatístico: Estatística descritiva uni e bivariada, histogramas, correlação, gráficos Chart.js, exportação em pdf

Perfis de utilizador: Anónimo / Registado / Administrador com controlo de acesso em cada endpoint da API.

3.Arquitetura do Sistema

3.1. Tecnologias

Por decisão do grupo, e após algumas indicações do docente Nathan Campos, o projeto foi implementado com as seguintes tecnologias:

Backend: PHP Vanilla

Frontend: JavaScript Vanilla (SPA)

Base de dados: MySQL (via MAMP/Apache)

Servidor web: Apache (MAMP)

Mapas: Leaflet.js, OpenStreetMap

Gráficos: Script Chart.js

Gestão de versões: GitHub

Mockups/Design: Figma

3.2. Estrutura de Ficheiros e Routing

A arquitetura de backend segue o padrão de single entry point com routing table: todos os pedidos HTTP chegam a `api/api.php`, que consulta uma tabela de routing e carrega exclusivamente o ficheiro PHP relevante para cada pedido. Esta solução resolve a ineficiência da segunda entrega, onde todos os handlers eram carregados em cada request.

Estrutura principal do repositório:

[Anexo 1]

3.3. Base de dados

O schema MySQL do Moodly foi desenvolvido para suportar todas as funcionalidades centrais da plataforma social, incluindo autenticação, interesses culturais, conexões entre utilizadores, mensagens, eventos geolocalizados, chats de grupo e ferramentas administrativas. A estrutura segue um modelo relacional normalizado, com utilização de foreign keys, constraints, índices e relações muitos-para-muitos.

Utilizador (usuario): Entidade central da plataforma responsável pelo armazenamento dos dados de autenticação e perfil. Contém identificador único, nome, email único, password com hash bcrypt, fotografia de perfil, tipo de perfil (user ou admin), estado de banimento

e data de registo. Foi criado um índice sobre o nome do utilizador para otimizar pesquisas textuais.

Interesse (interesse): Representa as categorias principais de interesses culturais da plataforma. O sistema limita estas categorias a três grupos base: Música, Jogos e Cinema & Séries.

Subinteresse (subinteresse): Representa interesses específicos pertencentes a uma categoria principal. Cada subinteresse está associado a uma categoria através de uma relação 1:N. Exemplos incluem géneros musicais, tipos de videojogos ou géneros cinematográficos.

Utilizador-Interesse (usuario_interesse): Tabela de junção responsável pela relação muitos-para-muitos entre utilizadores e subinteresses. Esta estrutura suporta diretamente o algoritmo de matching por afinidade implementado na plataforma. A combinação utilizador-subinteresse é única, impedindo duplicação de interesses.

Pedido de Conexão (pedido_conexao): Modela o sistema de ligações sociais entre utilizadores. Cada registo representa um pedido enviado entre dois utilizadores, contendo remetente, destinatário, estado do pedido (pendente, aceite ou recusado) e data de criação. A arquitetura foi simplificada para utilizar uma única tabela com estado inline, substituindo modelos mais complexos utilizados inicialmente.

Mensagens Diretas (post): Armazena mensagens privadas entre utilizadores conectados. Cada mensagem pertence a uma conexão específica e possui remetente, conteúdo, tipo de mensagem (text ou invite) e data de envio. O atributo post_tipo permite distinguir mensagens normais de convites para eventos enviados diretamente pelo chat.

Evento (evento): Entidade responsável pela gestão de eventos criados pelos utilizadores. Cada evento inclui criador, título, descrição, localização textual, coordenadas geográficas (latitude e longitude), data do evento e estado (activo ou cancelado). A presença de coordenadas GPS permite integração direta com Leaflet.js e funcionalidades SIG.

Convite de Evento (invite): Estrutura responsável pela participação de utilizadores em eventos. Cada convite associa um utilizador a um evento e contém um estado (pendente, confirmado, recusado ou cancelado). O criador do evento é automaticamente inserido como participante confirmado, enquanto os restantes utilizadores começam no estado pendente.

Chat de Grupo (group_post): Sistema de mensagens associado aos eventos. Cada mensagem pertence a um evento específico e a um utilizador participante, permitindo comunicação coletiva entre membros confirmados do evento.

Report de Utilizador (user_report): Implementa o sistema de denúncias da plataforma. Cada report armazena o utilizador denunciado, o utilizador denunciante, motivo da denúncia, estado do report (pending, reviewed ou dismissed) e data de criação. Esta funcionalidade é utilizada pelo backoffice administrativo.

Log Administrativo (admin_log): Regista ações realizadas por administradores da plataforma, incluindo o administrador responsável, ação executada, utilizador alvo e data da operação. Esta tabela permite rastreabilidade e auditoria administrativa.

O schema inclui ainda dados iniciais (seed data) utilizados para demonstração e testes. Foram definidas três categorias principais de interesses, Música, Jogos e Cinema & Séries, acompanhadas de múltiplos subinteresses iniciais. Apesar de os utilizadores poderem criar novos subinteresses dinamicamente através do sistema de tags, apenas as categorias principais permanecem fixas, garantindo consistência estrutural na plataforma.

4. Algoritmos e Estruturas de Dados

4.1. Algoritmo de Matching por Interesses

O algoritmo central do Moodly é o sistema de descoberta de utilizadores compatíveis. Em vez de apresentação aleatória, foi implementado um sistema de scoring com dois critérios de ordenação hierárquicos:

Critério 1. Prioridade de reciprocidade (sinal booleano de alta prioridade): utilizadores que já enviaram um pedido de ligação ao utilizador autenticado (e que estão à espera de resposta) aparecem sempre no topo da lista, garantindo que pedidos pendentes recebidos não passam despercebidos.

Critério 2. Pontuação de afinidade por sub-interesses (sinal contínuo): entre utilizadores sem pedido pendente, a ordenação é feita pelo número de sub-interesses partilhados, calculado por um LEFT JOIN duplo sobre a tabela usuario_interesse.

Passos da query SQL:

1. Para cada candidato, conta sub-interesses em comum via JOIN entre as entradas de usuario_interesse do utilizador autenticado e do candidato.
2. Deteta se o candidato enviou pedido pendente ao utilizador autenticado (sent_me_request = 1).
3. Exclui: utilizadores já conectados (qualquer estado), pedidos do próprio, utilizadores banidos e o próprio utilizador.
4. Ordena: ORDER BY sent_me_request DESC, score DESC — reciprocidade primeiro, afinidade depois.

Este algoritmo implementa um conceito de relevância semelhante a sistemas de recomendação com múltiplos sinais de ranking: um sinal booleano de alta prioridade combinado com um sinal contínuo de afinidade, numa estrutura de ranking dinâmico suportada pela base de dados relacional.

4.2. Dashboard Estatístico

O dashboard de estatísticas é exclusivo do perfil de administrador implementa análise descritiva uni e bivariada, calculada integralmente em PHP sem bibliotecas externas.

Estatística Descritiva Univariada (variável: número de conexões por utilizador):

- Média aritmética, Mediana, Moda (`array_count_values` + `arsort`)
- Desvio padrão populacional (raiz quadrada da variância)
- Mínimo, Máximo, Percentis P25 e P75 (acesso direto ao array ordenado)
- Intervalo de confiança a 95% da média: $IC = média \pm 1,96 \times \sigma/\sqrt{n}$
- Variáveis adicionais: mensagens por conversa e participantes por evento (média, máx, mín, sem mensagens)

Estatística Descritiva Bivariada:

Estudo da correlação entre número de interesses de um utilizador e o seu número de conexões aceites. Utilizadores agrupados em cinco classes (0 / 1–3 / 4–6 / 7–10 / 11+ interesses), com média de conexões calculada para cada classe. Resultado apresentado em gráfico de barras com interpretação automática ("Correlação positiva" vs. "Correlação não clara").

Histogramas:

Implementados para as distribuições de conexões por utilizador e mensagens por conversa. Os valores raw são retornados pelo PHP e os bins calculados em JavaScript com largura adaptativa: $\text{ceil}((\text{max} - \text{min} + 1) / n_bins)$. Os histogramas adaptam-se aos dados reais e não a intervalos fixos.

Interesse mais polarizador (indicador original):

Definido como o interesse com mais utilizadores, mas que gera proporcionalmente menos conexões. Score de polarização = $\text{utilizadores} / (\text{conexões} + 1)$. Um valor elevado indica um interesse popular que não traduz afinidade suficiente para gerar ligações.

Gráficos interativos e exportação PDF:

Todos os gráficos são renderizados com Chart.js com carregamento lazy (apenas quando a página de estatísticas é visitada). Tipos utilizados: linha, barras horizontais, barras verticais, donut e pizza. A exportação para PDF vetorial foi implementada com

window.print() e folha de estilos @media print dedicada, com mecanismo adicional via window.onbeforeprint / onafterprint para adaptar as cores internas dos gráficos Chart.js (canvas) antes da impressão.

4.3. Visualização de Informação Georreferenciada (SIG)

O requisito de informação georreferenciada foi implementado com Leaflet.js + tiles OpenStreetMap e geocodificação via API Nominatim (geocodificação direta e inversa).

Criação de eventos: o utilizador pode clicar no mapa para marcar a localização ou pesquisar um local por nome. A geocodificação inversa preenche automaticamente o campo de localização com o nome do lugar. O mapa de seleção é integrado diretamente no modal de criação.

Consulta de eventos: na página de Eventos, um botão toggle abre um mapa global que mostra todos os eventos confirmados do utilizador com marcadores. Ao clicar no detalhe de um evento, o botão "Ver localização no mapa" abre e faz scroll até ao mapa com o marcador do evento selecionado.

Ambas as bibliotecas (Leaflet CSS e JS) são carregadas de forma lazy, evitando impacto no tempo de carregamento inicial da SPA.

4.5. Algoritmos por Perfil de Utilizador

O sistema implementa três perfis com comportamentos e algoritmos distintos, com verificação de perfil no backend em cada request através das funções requireLogin() e requireAdmin():

Anónimo	Acesso apenas às páginas de login e registo. Qualquer endpoint protegido sem sessão retorna erro 401
Utilizador registado	Algoritmo de matching, sistema de conexões, mensagens diretas, chat de grupo, criação/participação de eventos
Administrador	Todas as funcionalidades do utilizador registado + painel de administração.

5. Funcionalidades Implementadas

5.1. Autenticação e Perfil

O sistema de autenticação inclui registo com validação de email único, login com verificação de hash bcrypt, gestão de sessões PHP server-side e logout com destruição de sessão. O perfil de utilizador inclui nome, email, foto de perfil (upload com redimensionamento), e seleção de sub-interesses organizados hierarquicamente por categorias (Música, Jogos, Cinema & Séries). As passwords são armazenadas com hash bcrypt, garantindo que o processo de autenticação é idêntico ao de produção.

5.2. Descoberta e Conexões

A página de Descoberta apresenta utilizadores ordenados pelo algoritmo de matching. Cada card exibe o nome, foto, interesses partilhados e score de afinidade. O utilizador pode enviar pedido de ligação diretamente do card. A pesquisa por nome permite encontrar utilizadores específicos. O sistema de conexões suporta os estados pendente, aceite e recusado, com notificação visual dos pedidos recebidos.

5.3. Mensagens Diretas

O chat direto está disponível entre utilizadores conectados. A interface segue um padrão visual semelhante ao Instagram (lista de conversas + painel de mensagens). A implementação em tempo real foi resolvida com polling manual (re-fetch após cada envio de mensagem), adequado para o contexto demonstrativo. Em produção, a solução ideal seria Server-Sent Events ou WebSockets.

5.4. Eventos

Os utilizadores podem criar eventos com título, data, localização georreferenciada (selecionada no mapa Leaflet) e convidar conexões. Os convites têm estados aceitos/recusados/pendente. Os participantes confirmados acedem a um chat de grupo do evento (group_post). Na página de Eventos, o mapa global exibe todos os eventos confirmados do utilizador com marcadores, e o botão de detalhe faz scroll e zoom até ao marcador do evento selecionado.

5.5. Administração e Backoffice

O painel de administração, exclusivo do perfil admin, inclui: gestão de utilizadores (listagem, ban/unban, promoção a admin), resolução de reports de utilizadores com histórico, e o dashboard estatístico completo. A verificação de perfil admin é feita no backend em cada request, sendo impossível contornar as restrições pelo frontend.

6. Evolução entre Entregas

O desenvolvimento do Moodly foi estruturado em três fases iterativas. Esta secção documenta as principais diferenças e evoluções entre cada entrega, evidenciando o progresso incremental da equipa.

Fase 1 para Fase 2: Principais Alterações

A transição entre a Fase 1 e a Fase 2 marcou a passagem de uma proposta conceptual para uma versão funcional inicial da plataforma. Durante a primeira entrega, o foco esteve centrado na definição do problema, estudo do público-alvo, levantamento de requisitos, construção de wireframes e planeamento da arquitetura geral do sistema. A segunda entrega representou o início da implementação prática da aplicação.

Ao nível da arquitetura e backend, foi implementada a infraestrutura principal da aplicação em PHP Vanilla, recorrendo a uma arquitetura baseada em single entry point através do ficheiro `api/api.php` e de uma routing table responsável pela distribuição dos pedidos HTTP. Foi igualmente definida a estrutura completa da base de dados MySQL, incluindo entidades, relações, constraints e índices necessários para suportar autenticação, conexões, eventos, mensagens e interesses culturais. Nesta fase foi também implementado o sistema de autenticação com sessões PHP server-side e hashing bcrypt para armazenamento seguro de passwords.

A API REST passou de uma estrutura conceptual para uma implementação funcional documentada em Markdown, contendo endpoints para autenticação, utilizadores, conexões, eventos e mensagens. Paralelamente, foi criado o ficheiro `populate.sql` com dados de teste realistas, incluindo utilizadores, conexões e eventos com coordenadas geográficas reais.

No frontend, a evolução foi igualmente significativa. Os wireframes e mockups produzidos na Fase 1 deram origem a uma SPA funcional desenvolvida em JavaScript Vanilla, com navegação dinâmica através de um router próprio e gestão global de estado via `App.state`. Foi também implementado o design system da plataforma, incluindo variáveis CSS, tipografia, componentes reutilizáveis e layout responsivo para dispositivos móveis.

As páginas de autenticação, login, registo, seleção de interesses e upload de fotografia de perfil, ficaram totalmente funcionais. Os mockups em Figma foram refinados e transformados numa interface navegável próxima do produto final.

Ao nível dos algoritmos e funcionalidades avançadas, a Fase 2 introduziu pela primeira vez o algoritmo de matching por afinidade, baseado em interesses partilhados e reciprocidade de pedidos pendentes. Foram ainda produzidos os diagramas UML do sistema e integrada a infraestrutura SIG através de Leaflet.js e da API Nominatim para geolocalização de eventos.

Fase 2 para Fase 3: Principais Alterações

A transição entre a Fase 2 e a Fase 3 correspondeu à evolução de um protótipo funcional para uma plataforma significativamente mais completa, otimizada e próxima de um cenário real de produção.

Ao nível da arquitetura e performance, a principal melhoria consistiu na refatoração do sistema de routing do backend. Na Fase 2, todos os handlers PHP eram carregados em cada request, independentemente da rota utilizada. Na versão final, o sistema passou a carregar exclusivamente o módulo necessário para cada pedido HTTP, reduzindo consumo de memória e melhorando a organização da aplicação.

A Fase 3 introduziu ainda várias funcionalidades novas. O dashboard estatístico passou a incluir estatística descritiva univariada e bivariada, histogramas adaptativos, cálculo de percentis, intervalos de confiança e correlação entre variáveis. Foi implementada exportação PDF vetorial do dashboard através de window.print() e adaptação dinâmica das cores dos gráficos Chart.js durante o processo de impressão.

Na componente UI/UX, a Fase 3 incluiu testes de usabilidade realizados com utilizadores reais através de protocolos think-aloud.

Na fase 3 a UI no geral também foi renovada para ficar mais perto do conceito inicial em questão de cores e botões.

Por fim, foram produzidos materiais finais de apresentação do projeto, incluindo poster promocional, slides, e vídeo promocional do Moodly.

7. UI/UX e Design System

O design do Moodly foi desenvolvido com foco no público-alvo de estudantes universitários (18–25 anos), privilegiando uma estética familiar ao ecossistema de aplicações sociais que este grupo utiliza quotidianamente.

Tipografia: Inter (Google Fonts). escolhida pela legibilidade em ecrã e pela neutralidade que permite que o conteúdo (utilizadores, interesses) seja o elemento central.

Paleta de cores: definida com variáveis CSS globais. cor de marca, cores de estado (success, danger, warning), fundos neutros e superfícies de card com contraste adequado para acessibilidade.

Componentes reutilizáveis: cards de utilizador, badges de interesse, modals, hamburger menu, chat list, botões de estado, todos definidos em CSS com variáveis do design system, garantindo consistência visual em toda a aplicação.

Responsividade: o layout adapta-se a mobile e desktop. O hamburger menu e a chat list com estilo Instagram foram implementados especificamente para mobile.

Processo iterativo: os mockups de alta fidelidade produzidos em Figma na Fase 2 foram validados com utilizadores reais na Fase 3, gerando iterações de design que melhoraram muito levemente a navegação

<https://www.figma.com/design/GfAitIJv9hktl0Lyk7gCV/Untitled?node-id=0-1&t=6rBPfa6JOa9Y8Yjh-1>

8. Testes de Usabilidade

Na Fase 3, foram realizados testes de usabilidade com utilizadores reais do público-alvo (estudantes universitários), seguindo um protocolo estruturado de avaliação com tarefas definidas e registo de observações.

[Anexo 2].

Metodologia: protocolo think-aloud como objetivo inicial, com tarefas definidas (Criação de um evento).

9. Análise Estatística

O projeto incorpora um módulo robusto de análise estatística descritiva e bivariada desenvolvido no backend (admin.php), cujos dados são consolidados e expostos visualmente através de uma dashboard administrativa (Statspage.js). Este componente foi desenhado para monitorizar o ecossistema da plataforma, avaliar o comportamento dos utilizadores e extrair métricas de engajamento através de abordagens estatísticas.

A análise subdivide-se nas seguintes frentes metodológicas:

Estatística Descritiva Univariada das Conexões: Através de funções agregadas e algoritmos nativos, o sistema calcula métricas de tendência central e de dispersão relativas ao número de conexões aceites por utilizador. São computadas dinamicamente a **média aritmética**, a **mediana**, a **moda**, o **desvio-padrão**, os valores **mínimo/máximo** e os **quartis (P25 e P75)**, oferecendo uma visão clara sobre a distribuição e assimetria do engajamento social na plataforma.

Análise Bivariada (Interesses vs. Conexões): O sistema realiza o cruzamento de variáveis categóricas organizando os utilizadores em intervalos quantitativos com base no número de subinteresses registados (ex: 0, 1-3, 4-6, 7-10, 11+). Para cada grupo, calcula-se a média de conexões ativas, permitindo avaliar estatisticamente se uma maior diversidade de interesses influencia o volume de laços sociais estabelecidos.

Métricas de Engajamento e Indicadores Específicos:

Interesses Populares e Agrupadores: Identificação das categorias e subinteresses com maior volume de entradas e cálculo da média de utilizadores por categoria.

Fator Polarizador: Algoritmo dedicado que calcula um rácio de densidade (utilizadores divididos por conexões geradas + 1), identificando quais os subinteresses que atraem muitos indivíduos mas que raramente resultam em conexões efetivas.

Atividade Temporal e Interações: Agrupamento e análise histórica de mensagens trocadas nos últimos 14 dias para mapear picos de atividade, além de médias de participação em eventos e mensagens por conversa.

Estes indicadores estatísticos fornecem à administração dados analíticos fundamentais para a moderação de conteúdos, suporte à decisão e otimização das dinâmicas de emparelhamento social da aplicação

10. Reflexão Crítica

A escolha de PHP Vanilla em vez de um framework como Laravel foi uma experiência de aprendizado significativa, não necessariamente intencional. A ausência de framework, decorrente de limitações do conhecimento inicial do grupo, já que o ensino de PHP na UC de Programação Web se manteve relativamente precário até ao final. Tivemos de implementar manualmente o routing, a gestão de sessões, a autenticação e a validação de dados. Estas são competências que ficam ocultas quando se utiliza um ORM ou middleware pronto, pelo que a experiência foi formativa apesar do custo em tempo.

A escolha de JavaScript Vanilla sem React ou Vue segue a mesma lógica: o padrão SPA com um único App.state e renderização via innerHTML demonstra compreensão profunda do DOM e do ciclo de vida de uma aplicação web.

Com mais tempo, as melhorias prioritárias seriam:

- Implementação de WebSockets para mensagens em tempo real.
- Algoritmo de matching mais sofisticado, talvez com pesos por categoria de interesse e decay temporal (interesses adicionados mais recentemente com maior peso).
- Deploy online numa plataforma gratuita (Railway, Render ou similar) para demonstração sem dependência do ambiente local.

11. Divisão de Tarefas

O projeto foi desenvolvido em colaboração contínua, com revisão cruzada de código entre todos os membros. A distribuição abaixo reflete as áreas de responsabilidade principal de cada elemento:

Deuwer Rabelais (20241521): Responsável pela base de dados e queries. Desenho e implementação do schema MySQL. Algoritmos de estatística descritiva em PHP (média, mediana, moda, desvio padrão, percentis, IC95%, bivariada, polarizador). Ficheiro populate.sql com dados de teste. Relatórios e design language do UI. Slides.

Isimar Quixito (20242038): Responsável por grande parte do frontend e mockups. páginas de autenticação (login, registo, interesses, foto de perfil), design system (CSS completo com variáveis, componentes reutilizáveis). Implementação do Inter font, brand colors e responsividade mobile.

Miguel Almeida (20240206): Maioria do Backend. e Eventos (convites, aceitação/recusa, cancelamento), mapa Leaflet com geocodificação Nominatim, modals de detalhe com participantes e botão de localização. Diagramas UML (modelo de domínio e casos de uso). Documentação REST API em Markdown e Word, Relatórios.

Vivandro Kambanza (20241805): Edição de slides e vídeos. Página de estatísticas (Statspage.js com Chart.js, histogramas, correlação, gráficos). Adaptação mobile com hamburger menu, chat list estilo instagram, layout responsivo em todos os ecrãs.

A coordenação entre membros foi feita via WhatsApp e Discord. O controlo de versões foi gerido com GitHub.

12. Conclusão

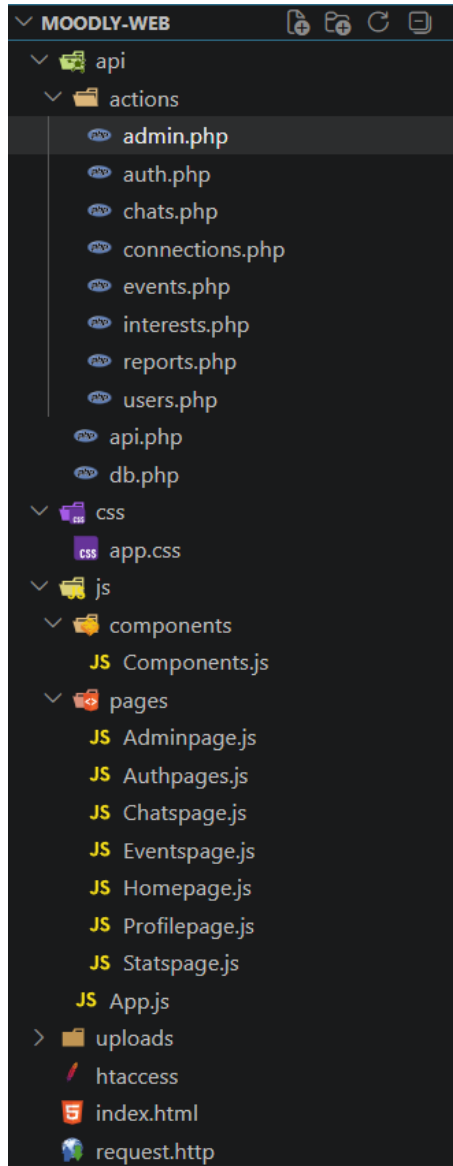
O desenvolvimento do Moodly permitiu aplicar de forma integrada os conhecimentos de todas as Unidades Curriculares do 4.º semestre num contexto real e motivante. A implementação do algoritmo de matching, dos cálculos de estatística descritiva, dos histogramas, da análise bivariada e da visualização georreferenciada demonstram que os fundamentos de AED, Estatística e SIG são a base que torna possível transformar dados em informação útil para a tomada de decisão.

O projeto cumpre os requisitos enunciados no briefing do Projeto de Desenvolvimento Web: implementação em PHP com base de dados relacional (MySQL), backoffice de administração com dados estatísticos, integração com sistema de informação geográfica (Leaflet + Nominatim), análise estatística dinâmica, algoritmo específico para a lógica de negócio (matching por afinidade), e estudo de UI/UX validado com testes de utilizadores reais. As ferramentas obrigatórias GitHub e Figma foram utilizadas ao longo de todo o projeto.

A experiência de desenvolver uma aplicação web de raiz, sem framework de backend nem de frontend, foi desafiante mas formativa. Obrigou a equipa a compreender profundamente cada componente do sistema, desde o routing HTTP até aos algoritmos de ordenação e agregação estatística, passando pela gestão de estado no frontend e pela segurança no backend.

13. Anexos

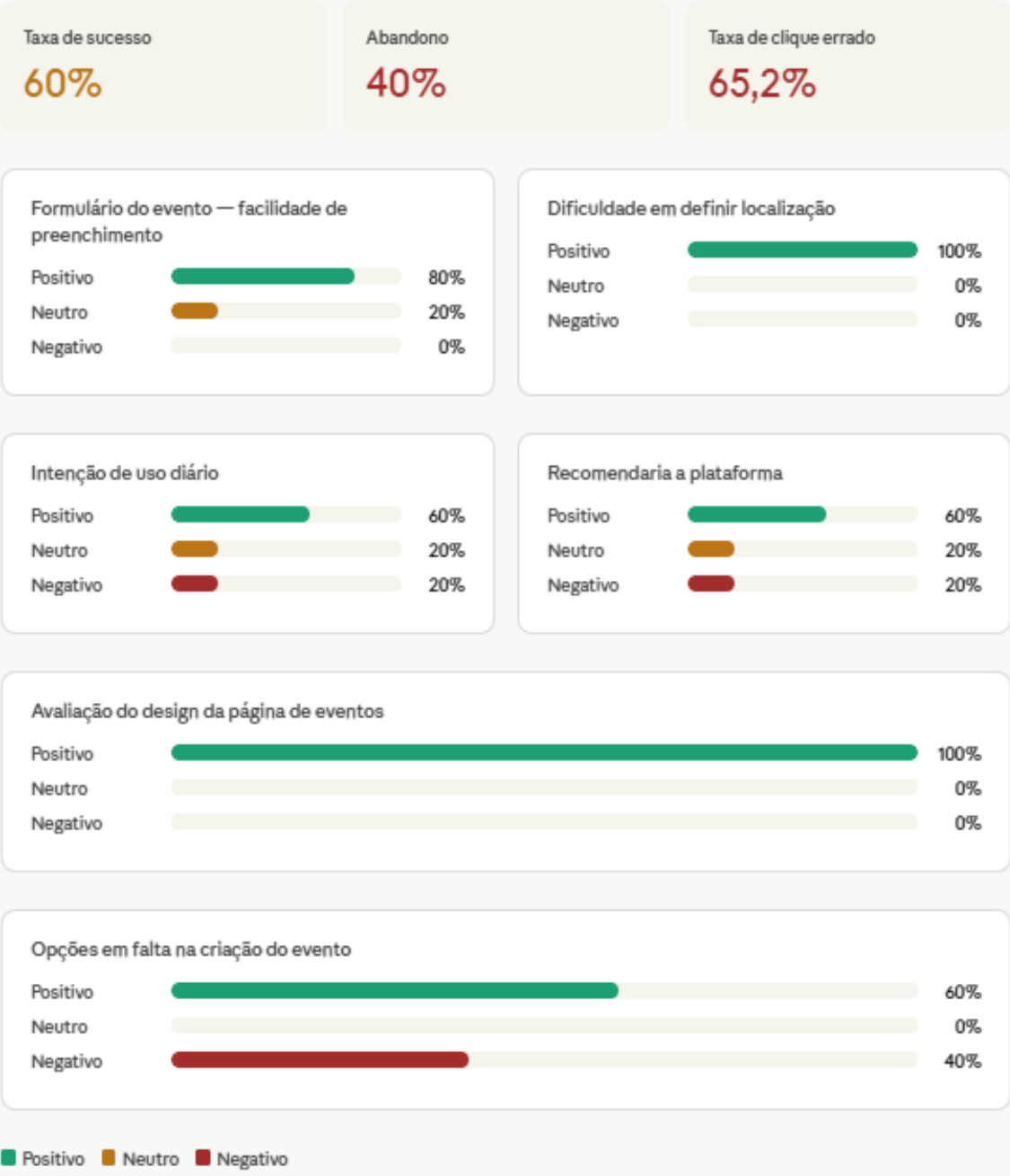
[Anexo 1]



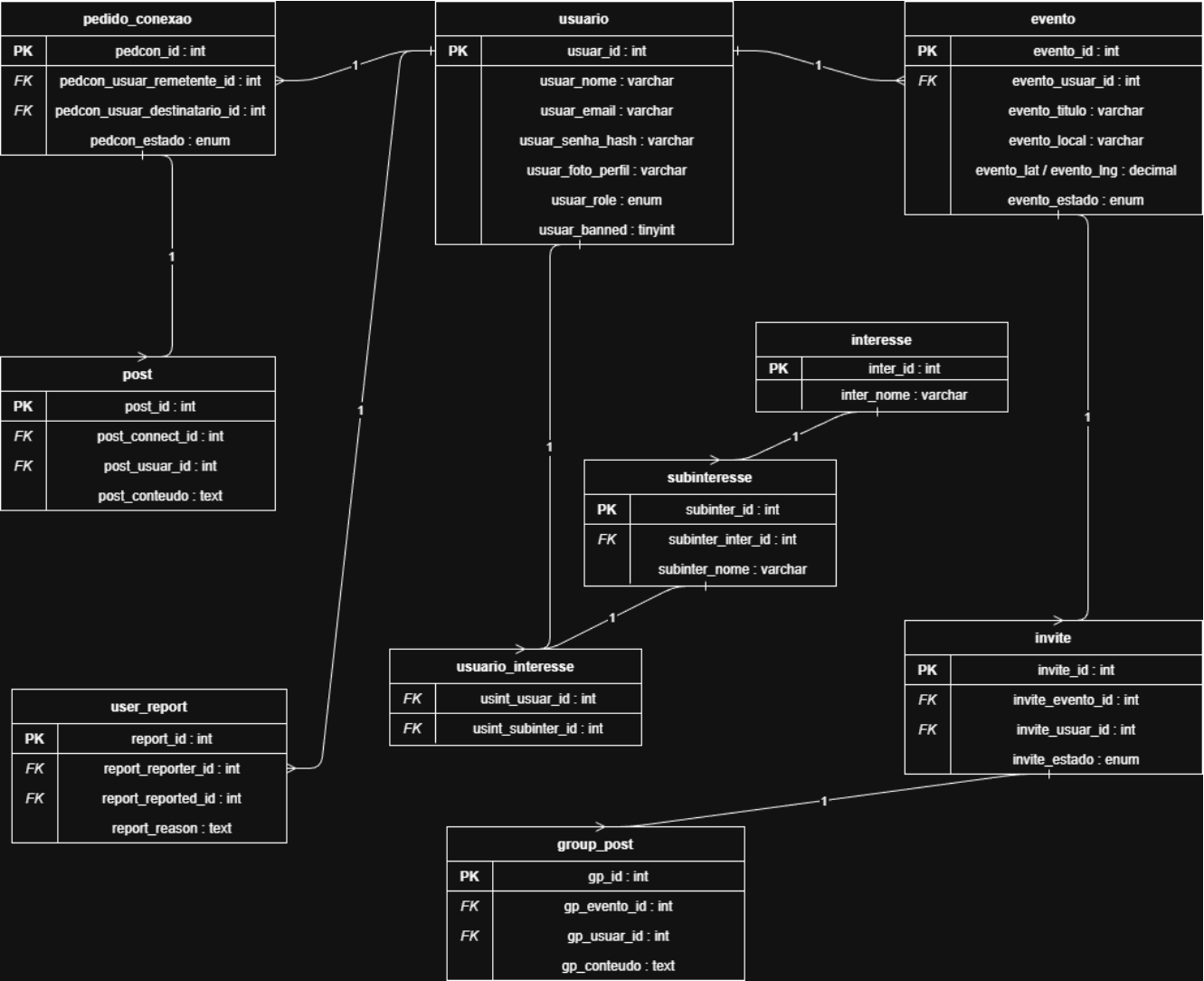
[Anexo 2]

Moodly — relatório de investigação UI

Teste de protótipo · Baseado em ecrã · 5 respostas · Duração média 33,7s



[Anexo 3]



[Anexo 4]

